

CHALLENGE 5: Validazione contenuto articolo non presente o non corretta

Vulnerabilità:

L'applicazione permette la creazione di articoli con un editor HTML. Tuttavia, non esiste alcuna validazione o sanificazione del contenuto inviato.

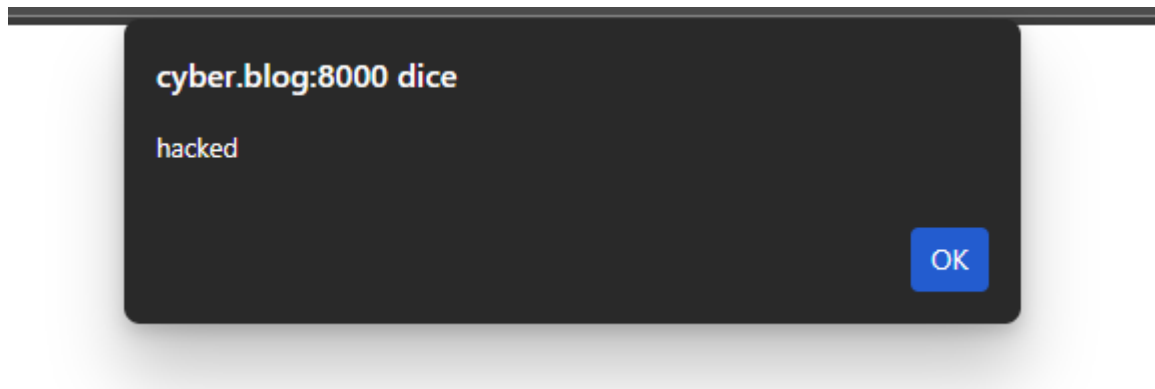
Un attaccante può intercettare la richiesta HTTP (es. con Burp Suite), modificare il campo body e inserire script malevoli.

Payload utilizzato:

```
<script>alert('hacked')</script>
```

Attacco eseguito:

- 1) Creazione di un articolo tramite il form normale
 - 2) Intercettazione della richiesta con Burp Suite
 - 3) Sostituzione del body con il payload malevolo
 - 4) Invio della richiesta modificata
- Apertura dell'articolo e esecuzione dello script...



Mitigazione:

Soluzione implementata:

Come richiesto dal PDF, ho applicato due livelli di protezione:
Filtro del testo prima del salvataggio

Nel controller **ArticleController.php**, il campo body viene sanificato con `strip_tags()`, consentendo solo tag HTML sicuri (testo formattato, liste, titoli, ecc.).

Escapamento in fase di visualizzazione

Nella vista **show.blade.php**, il contenuto viene mostrato con `{{ }}` invece di `{!! !!}`, impedendo l'esecuzione di qualsiasi codice HTML/JS.

In ArticleController.php:

```
$cleanBody = strip_tags($request->body,  
'<p><br><strong><em><ul><ol><li><h1><h2><h3><h4><h5><h6><blockquote><code><pre>');  
// ...  
'body' => $cleanBody,
```

In show.blade.php:

```
<p>{{ $article->body }}</p>
```

La vulnerabilità Stored XSS è stata eliminata grazie a:

Sanificazione rigorosa in input

Escapamento automatico in output

L'applicazione ora rispetta i principi di sicurezza per la gestione di contenuti utente.